

KREACIJSKI PATERNI U SISTEMU

1. Objašnjenje potrebe za uvođenjem kreacijskih paterna

U postojećem sistemu, direktno kreiranje objekata (korištenjem ključne riječi *new*) unutar kontrolera ili drugih servisa dovodi do visoke spregnutosti (*tight coupling*). Ako se promijeni konstruktor klase Student, Profesor ili ZahtjevZaDokument, moramo mijenjati kod na svim mjestima gdje se ti objekti kreiraju. Kako bismo izolovali logiku instanciranja i učinili sistem fleksibilnijim, u arhitekturu uvodimo dva kreacijska paterna: Factory Method (Fabrička metoda) i Singleton (Jedinstveni primjerak).

A) Factory Method Pattern (Patern Fabrička metoda)

- Zašto je potreban? Sistem podržava različite vrste korisnika (Student, Profesor, Administrator, Studentska služba) koji svi nasljeđuju baznu klasu Korisnik. Trenutno, kontroleri moraju eksplicitno znati koju klasu instanciraju na osnovu uloge, što stvara uslovna grananja u kodu. Također, inicijalizacija specifičnih korisnika zahtijeva kreiranje njihovih unutrašnjih kolekcija (npr. listi prijava ispita ili predanih zadaća).
- Kako rješava problem? Uvodimo apstraktnu klasu KorisnikFactory sa fabričkom metodom kreirajKorisnika(). Konkretno fabrike (StudentFactory, ProfesorFactory) nasljeđuju ovu klasu i preuzimaju na sebe kompletnu kompleksnost inicijalizacije i alokacije objekata. Kontroler u ovom slučaju samo poziva odgovarajuću fabriku preko njenog interfejsa, uopšte ne razmišljajući o unutrašnjim detaljima kreiranja konkretnog objekta.

B) Singleton Pattern (Patern Jedinstveni primjerak)

- Zašto je potreban? U sistemu postoji potreba za centralizovanim komponentama koje upravljaju globalnim stanjima, postavkama ili resursima (kao što su trenutna akademska godina, statusi rokova, konfiguracije sistema ili moduli za logovanje akcija). Kreiranje više instanci

ovakvih objekata kroz aplikaciju nepotrebno bi trošilo memoriju i stvorilo rizik od inkonzistentnosti podataka i konflikata u sesijama.

- Kako rješava problem? Patern osigurava da klasa SistemKonfiguracija ima samo jednu jedinstvenu instancu u čitavoj aplikaciji tokom njenog životnog ciklusa. Kroz privatni konstruktor onemogućava se eksterni poziv sa new, dok se globalna i sigurna pristupna tačka toj jedinstvenoj instanci pruža putem statičke metode getInstance().